

BÁO CÁO DỰ ÁN

Xây dựng Game Caro (Gomoku) tích hợp AI

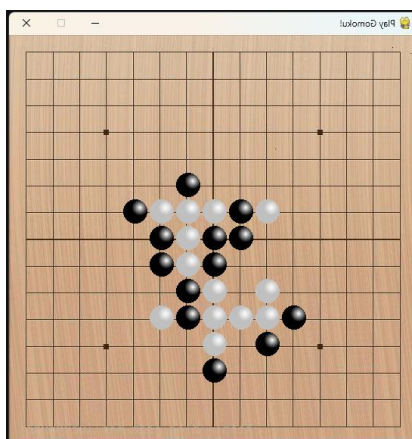
Sinh viên: Lê Đỗ Tùng Dương, Đỗ Duy Hưng, Hoàng Ngọc Đăng

Mã sinh viên: 24022304-24022346-24022280

Trường: Đại học Công nghệ - ĐHQGHN

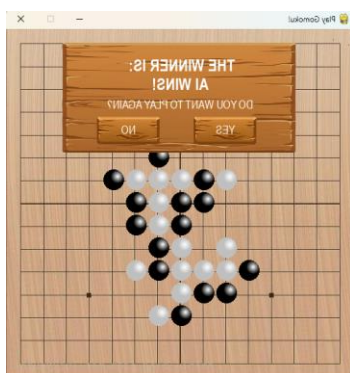
1. Mô tả bài toán cờ Caro

Bài toán xây dựng một chương trình máy tính có khả năng chơi cờ Caro (Gomoku) tự động. Đây là một dạng bài toán tìm kiếm trên không gian trạng thái cực lớn, nơi AI phải tính toán để đưa ra nước đi tối ưu nhằm chiến thắng đối thủ con người hoặc AI khác.



2. Luật chơi và điều kiện thắng

Bàn cờ có kích thước 15x15. Hai người chơi thay phiên nhau đặt quân Đen và Trắng. Điều kiện thắng: Người chơi nào tạo được một hàng liên tục gồm đúng 4 quân cờ cùng màu (ngang, dọc hoặc chéo) sẽ giành chiến thắng.



3. Cách biểu diễn trạng thái bàn cờ

Trạng thái bàn cờ được biểu diễn bằng một mảng hai chiều boardMap[15][15].

- 0: Ô trống.
- 1: Quân cờ của AI (Người chơi Max).
- -1: Quân cờ của người chơi (Người chơi Min).

```
class GomokuAI:
    def __init__(self, depth=5):
        self.depth = depth
        self.boardMap = [[0] * SIZE for _ in range(SIZE)]

        self.currentRow = -1
        self.currentCol = -1
        self.nextBound = {}      # set of (row,col) candidate moves
        self.boardValue = 0

        self.turn = 0           # which colour the human chose; set by UI
        self.lastPlayed = 0     # 1 or -1, last piece placed on the real board
        self.emptyCells = SIZE * SIZE

        self.patternDict = utils.createPatternDict()
        self.zobristTable = utils.createZobristTable(SIZE)
        self.rollingHash = 0
        self.transpositionTable = {}
```

4. Cách sinh nước đi hợp lệ

Thay vì xét toàn bộ các ô trống trên bàn cờ (độ phân nhánh 225), chương trình sử dụng kỹ thuật Bound Search. Các nước đi ứng viên chỉ được sinh ra trong phạm vi bán kính 1 hoặc 2 ô xung quanh các quân cờ đã có trên bàn cờ. Kỹ thuật này giúp giảm đáng kể thời gian tính toán của thuật toán tìm kiếm.

```
newBound = dict(bound)
self.updateBound(row, col, newBound)
ev = self.minimax(depth - 1, newEval, newBound, alpha, beta, False)

self.boardMap[row][col] = 0
self.rollingHash ^= self.zobristTable[row][col][0]

if ev > maxEval:
    maxEval = ev
    if depth == self.depth:
        self.currentRow, self.currentCol = row, col
        self.boardValue = newEval
        self.nextBound = newBound
```

```

def updateBound(self, row, col, bound):
    """Add empty neighbours of (row,col) to the candidate set."""
    bound.pop((row, col), None)
    for drow, dcol in DIRECTIONS:
        for sign in (1, -1):
            r, c = row + sign * drow, col + sign * dcol
            if self.isOnBoard(r, c) and self.boardMap[r][c] == 0 and (r, c) not in bound:
                bound[(r, c)] = 0

```

5. Cách kiểm tra trạng thái kết thúc

Hàm kiểm tra trạng thái sẽ quét theo 4 hướng (ngang, dọc, 2 đường chéo) từ nước đi cuối cùng. Nếu phát hiện chuỗi 5 quân cùng màu, trận đấu kết thúc. Nếu bàn cờ không còn ô trống, kết quả là hòa.

```

# — Game result —————

def checkResult(self):
    if (self.lastPlayed != 0
        and self.isOnBoard(self.currentRow, self.currentCol)
        and self.check4(self.currentRow, self.currentCol, self.lastPlayed)):
        return self.lastPlayed
    if self.emptyCells <= 0:
        return 0
    return None

def getWinner(self):
    r = self.checkResult()
    if r == 1: return "AI wins!"
    if r == -1: return "Player wins!"
    if r == 0: return "It's a draw!"
    return ""

```

6. Thuật toán Minimax đã cài đặt

Chương trình sử dụng thuật toán Minimax đệ quy để duyệt cây trò chơi. Tại mỗi bước, AI giả định rằng cả hai bên đều chơi tối ưu để chọn ra nước đi mang lại lợi ích lớn nhất. Kết hợp cả Minimax và Alpha-Beta trong hàm minimax().

7. Thuật toán Alpha-Beta đã cài đặt

Kỹ thuật cắt tỉa Alpha-Beta được áp dụng để loại bỏ các nhánh cây không cần thiết, giúp AI có thể duyệt sâu hơn trong cùng một khoảng thời gian so với Minimax thuần túy. Ngoài ra, chương trình còn sử dụng Zobrist Hashing và Transposition Table để tránh tính toán lại các trạng thái đã lặp lại.

```
# — Minimax —

def minimax(self, depth, boardValue, bound, alpha, beta, isMaximizing):
    if depth == 0 or not bound:
        return boardValue

    cached = self.transpositionTable.get(self.rollingHash)
    if cached is not None and cached[1] >= depth:
        return cached[0]

    if isMaximizing:
        maxEval = -math.inf
        for row, col in self._sortedCandidates(bound):
            delta = self._scoreDelta(row, col, 1)
            newEval = boardValue + delta

            self.boardMap[row][col] = 1
            self.rollingHash ^= self.zobristTable[row][col][0]

            if self.check4(row, col, 1):
                self.boardMap[row][col] = 0
                self.rollingHash ^= self.zobristTable[row][col][0]
                if depth == self.depth:
                    self.currentRow, self.currentCol = row, col
                    self.boardValue = newEval
                    self.nextBound = dict(bound)
                    return math.inf

            newBound = dict(bound)
            self.updateBound(row, col, newBound)
            ev = self.minimax(depth - 1, newEval, newBound, alpha, beta, False)

            self.boardMap[row][col] = 0
            self.rollingHash ^= self.zobristTable[row][col][0]

            if ev > maxEval:
                maxEval = ev
                if depth == self.depth:
                    self.currentRow, self.currentCol = row, col
                    self.boardValue = newEval
                    self.nextBound = newBound

            alpha = max(alpha, ev)
            if beta <= alpha:
                break

        utils.updateTranspositionTable(self.transpositionTable, self.rollingHash, maxEval, depth)
        return maxEval

    else:
        minEval = math.inf
```

```

def minimax(self, depth, boardValue, bound, alpha, beta, isMaximizing):
    if beta <= alpha:
        break

    utils.updateTranspositionTable(self.transpositionTable, self.rollingHash, maxEval, depth)
    return maxEval

else:
    minEval = math.inf
    for row, col in self._sortedCandidates(bound):
        delta = self._scoreDelta(row, col, -1)
        newEval = boardValue + delta

        self.boardMap[row][col] = -1
        self.rollingHash ^= self.zobristTable[row][col][1]

        if self.check4(row, col, -1):
            self.boardMap[row][col] = 0
            self.rollingHash ^= self.zobristTable[row][col][1]
            if depth == self.depth:
                self.currentRow, self.currentCol = row, col
                self.boardValue = newEval
                self.nextBound = dict(bound)
            return -math.inf

        newBound = dict(bound)
        self.updateBound(row, col, newBound)
        ev = self.minimax(depth - 1, newEval, newBound, alpha, beta, True)

        self.boardMap[row][col] = 0
        self.rollingHash ^= self.zobristTable[row][col][1]

        if ev < minEval:
            minEval = ev
            if depth == self.depth:
                self.currentRow, self.currentCol = row, col
                self.boardValue = newEval
                self.nextBound = newBound

        beta = min(beta, ev)
        if beta <= alpha:
            break

    utils.updateTranspositionTable(self.transpositionTable, self.rollingHash, minEval, depth)
    return minEval

```

8. Hàm đánh giá trạng thái (Heuristic)

Hàm đánh giá sử dụng hệ thống Pattern Matching (so khớp mẫu). Mỗi chuỗi quân cờ (như 4 quân hỏ 2 đầu, 3 quân bị chặn...) sẽ được gán một điểm số tương ứng. Tổng điểm của bàn cờ là hiệu số giữa điểm của AI và điểm của người chơi.

```
def createPatternDict():
    p = 1 # placeholder; evaluator substitutes actual player value

    patterns = {
        # 4 in a row = instant win
        (p, p, p, p): 1_000_000,

        # Open 3 - one away from a forced win
        (0, p, p, p, 0): 100_000,

        # Broken 3 - skip in the middle, still very dangerous
        (p, p, 0, p): 50_000,
        (p, 0, p, p): 50_000,

        # Half-open 3 - one end blocked
        (0, p, p, p, -1): 10_000,
        (-1, p, p, p, 0): 10_000,

        # Open 2
        (0, p, p, 0): 1_000,
        (0, p, 0, p, 0): 1_000,

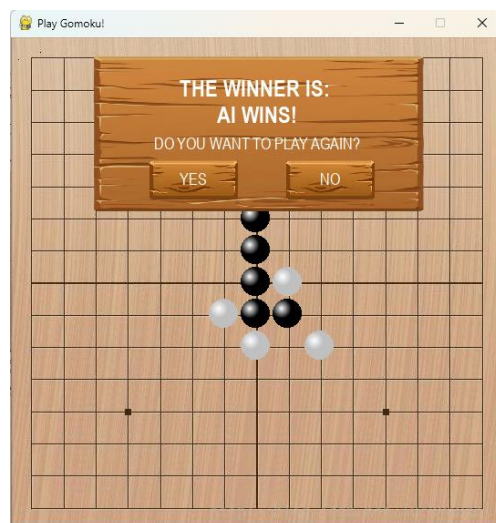
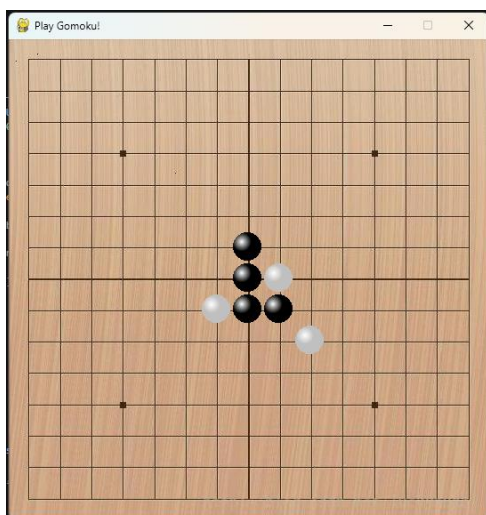
        # Half-open 2
        (0, p, p, -1): 100,
        (-1, p, p, 0): 100,

        # Separated pair
        (p, 0, 0, p): 500,
    }
    return patterns
```

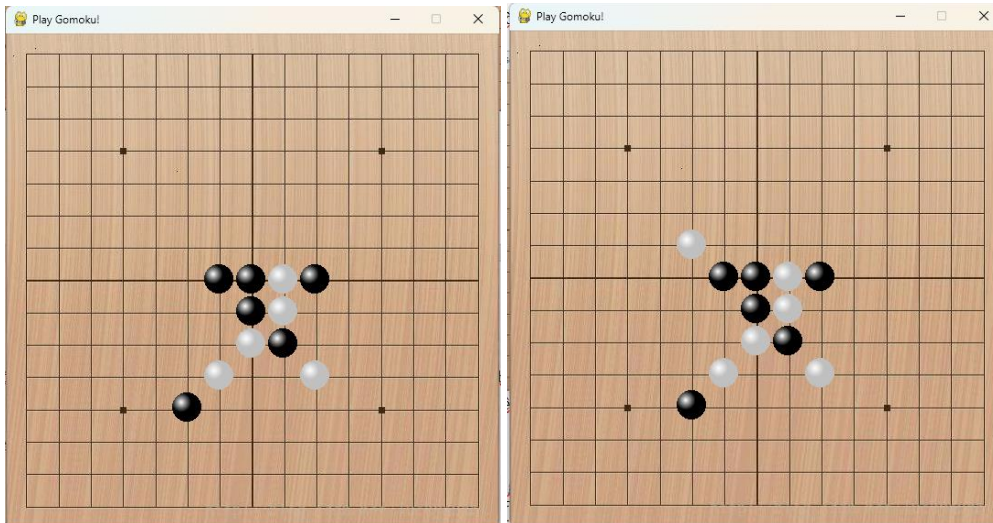
9. Thiết kế các trạng thái thử nghiệm

Các kịch bản kiểm thử bao gồm:

- 1. Tấn công: AI nhận diện được chuỗi 4 quân để kết thúc trận đấu.
(AI quân đen, người chơi quân trắng)



- 2. Phòng thủ: AI nhận diện được chuỗi 3 hoặc 4 quân của đối thủ để chặn kịp thời.
(AI quân trắng, Người chơi quân đen)



- 3. Hiệu năng: Kiểm tra thời gian phản hồi khi tăng độ sâu tìm kiếm.

- Độ sâu: 4

```
AI move took 2.08s
AI places at row=6, col=5
```

- Độ sâu : 5

```
AI move took 2.53s
AI places at row=7, col=8
```

10. Bảng kết quả thực nghiệm

Độ sâu (Depth)	Số trạng thái (Ước tính)	Thời gian trung bình (s)	Đánh giá
3	~5.000	0.2s	Rất nhanh
4	~45.000	0.8s	Thông minh cơ bản
5	~400.000	2.5s	Khá thông minh

11. Nhận xét về số trạng thái và thời gian chạy

Số lượng trạng thái tăng theo hàm mũ. Tuy nhiên, việc áp dụng Zobrist Hashing và Alpha-Beta đã giúp AI duy trì tốc độ phản hồi dưới 3 giây ở độ sâu 5, đủ tốt cho trải nghiệm người dùng.

12. Nhận xét về ảnh hưởng của độ sâu tìm kiếm

Độ sâu 5 giúp AI có cái nhìn xa hơn, phát hiện được các tình huống bẫy cờ phức tạp mà ở độ sâu 3 AI thường bỏ qua.

13. Ưu điểm và hạn chế

Ưu điểm: Giao diện trực quan, AI khá thông minh, thuật toán tối ưu.

Hạn chế: Chưa áp dụng các thuật toán nâng cao hơn như Monte Carlo Tree Search hay Neural Networks.

14. Tài liệu tham khảo

1. Bài giảng Cơ sở trí tuệ nhân tạo - ĐH Công nghệ.
2. Repository tham khảo: <https://github.com/husus/gomokuAI-py>

Tham khảo từ repository:

- Cách tổ chức project Python cho game Gomoku/Caro.
- Cách tách phần giao diện, phần xử lý game và phần AI.
- Cách biểu diễn bàn cờ bằng mảng hai chiều.
- Cách kiểm tra nước đi hợp lệ và kiểm tra thắng/thua.
- Cách xây dựng giao diện đơn giản bằng pygame.